10PEARLS

PEARLS AQI PREDICTOR

End-to-End Air Quality Index Forecasting &
Explainable Machine Learning Platform

FINAL PROJECT REPORT

Prepared By
Noor Fatima

Data Science / Machine Learning
Lahore • Islamabad • Faisalabad

1. Project Overview

Pearls AQI Predictor is designed as a modular forecasting platform. The project separates data acquisition, validation, feature engineering, model training, explainability, application serving, visualization, and testing into dedicated areas of the repository.

The central workflow is: external telemetry → validation → feature engineering → processed features → model training → model artifacts → prediction → explainability → API → dashboard.

The documented repository structure includes feature_pipeline, training_pipeline, models, explainability, app, utils, data, tests, dags, reports, and GitHub workflow configuration. This structure supports separation of concerns and makes individual pipeline stages easier to maintain and test.

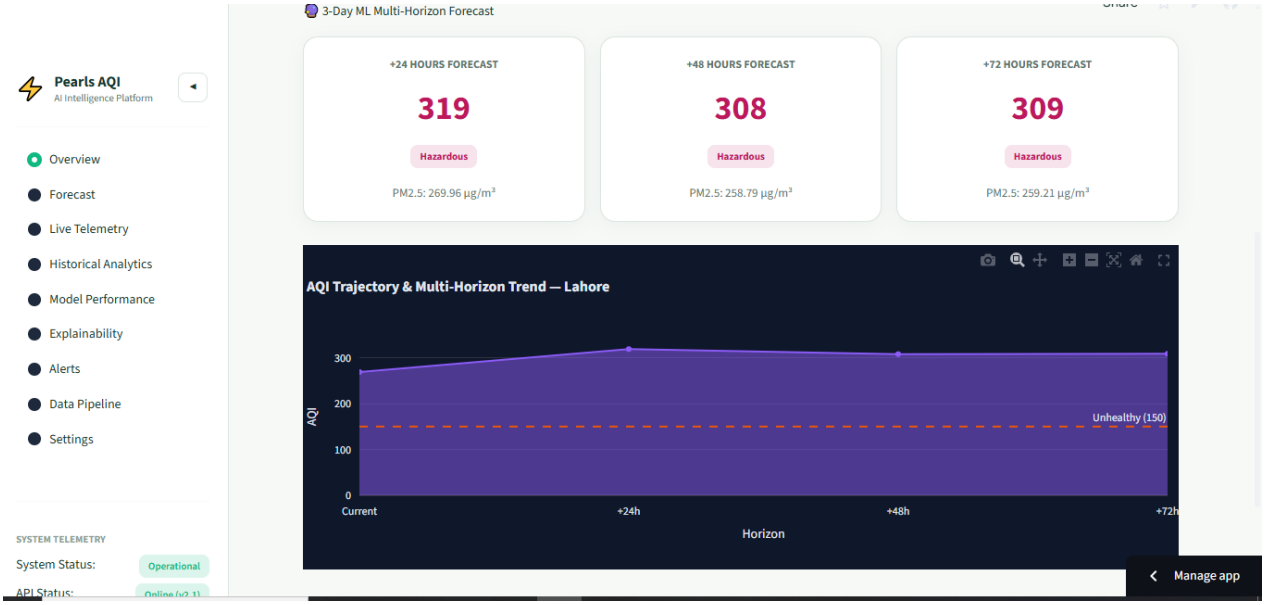


Figure1: End-to-end data-to-prediction workflow

2. Problem Statement

Air quality varies over time because pollutant concentrations interact with weather conditions, temporal patterns, and local environmental behavior. Monitoring current AQI is valuable, but forecasting future AQI provides additional decision-support value.

The project addresses the forecasting problem by combining pollutant and meteorological observations with temporal and historical features. Machine-learning models then estimate future AQI over multiple horizons.

A second problem is model interpretability. A prediction is more useful when users can understand which input characteristics contributed to it. The project therefore incorporates SHAP-based explainability alongside prediction.

Core engineering objective: transform external environmental telemetry into repeatable, testable, accessible, and explainable AQI forecasts.

3. Objectives & Scope

The supplied project brief describes an end-to-end AQI prediction system, automated feature and training pipelines, an interactive web application, explainability, alerts/analytics, and detailed documentation.

The project material documents these implemented areas:

• Real-time air-quality and meteorological ingestion.

• Feature engineering with temporal, lag, and rolling-window signals.

• Forecasting support for Lahore, Islamabad, and Faisalabad.

• 24-hour, 48-hour, and 72-hour forecasting as documented.

• Multiple machine-learning regressors, an MLP neural network, and ensemble modeling.

• SHAP-based feature contribution analysis.

• Flask REST API for authentication, telemetry, and forecasting.

• Streamlit dashboard for interactive monitoring and forecasts.

• Automated tests and pipeline/orchestration configuration.

• Git/GitHub repository organization and release management.

4. Technology Stack

Layer	Technology	Purpose
Language	Python	Core implementation
Data	Pandas, NumPy, PyArrow / FastParquet	Processing and storage
ML	Scikit-Learn	Model development and evaluation
Models	Ridge, MLP, Random Forest, Stacking	Forecasting approaches documented
Persistence	Joblib	Model artifact serialization
XAI	SHAP	Feature contribution analysis
API	Flask, Flask-CORS	REST service layer
UI	Streamlit, Plotly, custom CSS	Interactive dashboard
Database	SQLite	Persistent application/session data
Telemetry	OpenAQ v3, Open-Meteo	Environmental inputs
Testing	Pytest	Automated QA
Automation	GitHub Actions, Apache Airflow	Workflow automation/orchestration
VCS	Git, GitHub	Source control and delivery

The stack forms a practical Python-based architecture: APIs supply environmental observations, data libraries prepare features, machine-learning components generate forecasts, Flask provides a service boundary, Streamlit provides the analytical interface, and SHAP adds interpretability.

5. System Architecture

The architecture follows a layered pipeline. Each stage has a defined responsibility and passes structured information to the next stage.

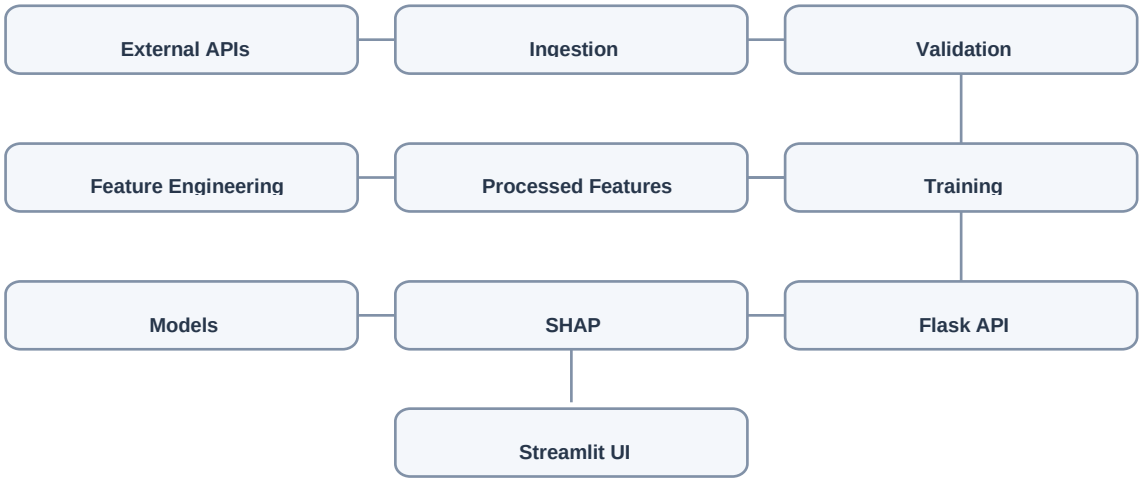


Figure 2 — High-level architecture of the documented system.

▪ **Telemetry layer:** external air-quality and weather services provide raw observations.

• **Data pipeline:** ingestion, validation, and feature engineering convert raw observations into model-ready data.

• **Training layer:** candidate models are trained and evaluated, with artifacts stored for inference.

• **Explainability layer:** SHAP analysis provides feature-level interpretation.

• **Application layer:** Flask exposes backend services and Streamlit presents interactive results.

6. Data Collection & Telemetry

The documented implementation uses OpenAQ v3 for air-quality telemetry and Open-Meteo for meteorological information. The project describes pollutant inputs including PM2.5, PM10, NO2, SO2, CO, and O3, together with temperature, humidity, wind speed, wind direction, and surface pressure.

Category	Documented inputs
Pollutants	PM2.5, PM10, NO2, SO2, CO, O3
Weather	Temperature, humidity, wind speed, wind direction, surface pressure
Cities	Lahore, Islamabad, Faisalabad
Sources	OpenAQ v3 and Open-Meteo

The ingestion layer is separated from validation and feature engineering so that downstream stages can operate on structured and quality-checked data.

7. Data Quality & Feature Engineering

Feature engineering converts raw environmental observations into predictive signals. The project documentation identifies cyclical time transformations, rolling-window statistics, and lag indicators.

- **Cyclical time features:** sine/cosine representations of time variables help model repeating temporal patterns without treating the end and beginning of a cycle as far apart.
- **Rolling statistics:** 6-hour, 12-hour, and 24-hour means, maxima, and standard deviations summarize recent pollution behavior.
- **Lag indicators:** previous observations provide historical context and allow models to capture persistence and short-term dynamics.
- **Data safeguards:** missing-value handling, out-of-range clipping, and schema validation are documented as quality controls.

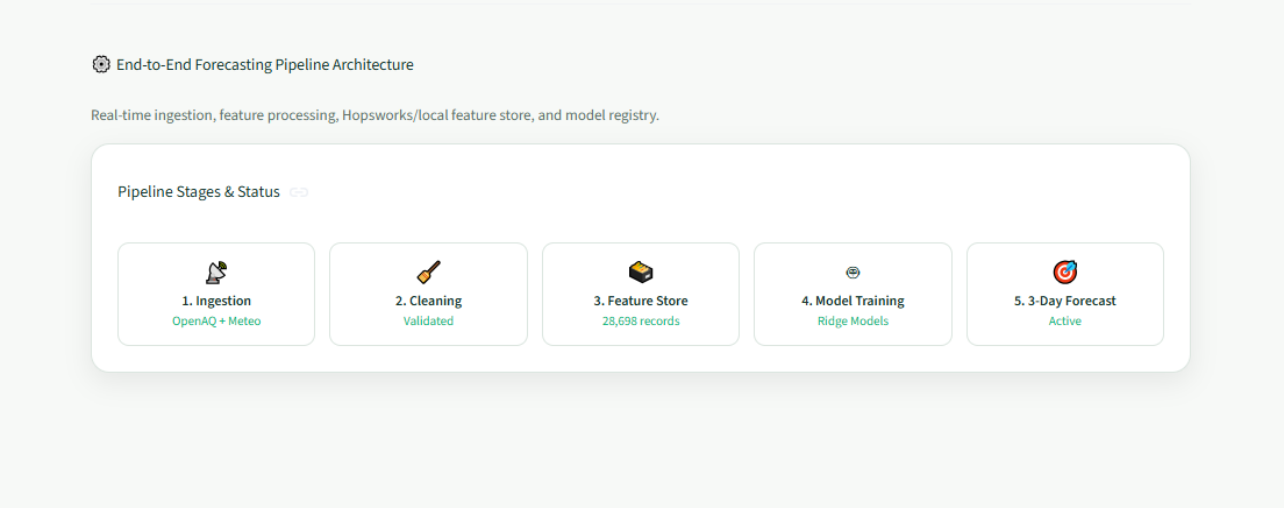
The documented feature engineering module is organized around the feature_pipeline area, with utility support for data quality and feature storage. The processed feature dataset is documented as data/processed/model_features_3cities.csv.

8. Machine Learning Pipeline

The machine-learning workflow follows a repeatable sequence: prepare historical features and targets, train candidate models, evaluate them using forecasting metrics, select appropriate model artifacts, serialize trained models, and use the selected artifacts for prediction.

Stage	Purpose
Historical data preparation	Create a learning dataset from processed environmental features
Training	Fit candidate forecasting models
Evaluation	Compare model behavior using RMSE, MAE and R^2 where available
Selection	Identify appropriate models for documented forecasting tasks
Persistence	Store trained model artifacts using Joblib
Inference	Load model artifacts and generate forecast outputs

The documented training script is located under training_pipeline/train_3cities.py, while model artifacts are organized under models/3cities/.



9. Models & Forecasting Horizons

The project documentation describes multiple model families so that the forecasting system can capture both linear relationships and more complex non-linear interactions.

Model	Role described in project material
Ridge / Linear Regression	Low-latency regression approach for forecasting
MLP Neural Network	Captures more complex non-linear interactions
Random Forest	Tree-based candidate model for experimentation
Stacking Ensemble	Combines multiple predictive learners into an ensemble

The documented horizons are 24 hours, 48 hours, and 72 hours. The project material associates different model approaches with different horizons; however, exact model-to-horizon mapping should be read from the final implementation when this report is used for a formal technical audit.

Horizon	Selected Model	RMSE	MAE	R^2
24 hours	Verify from implementation	—	—	—
48 hours	Verify from implementation	—	—	—
72 hours	Verify from implementation	—	—	—

10. Model Evaluation

Model evaluation is essential because forecasting quality cannot be established from architecture alone. The project brief identifies RMSE, MAE, and R^2 as evaluation measures.

• **RMSE:** emphasizes larger prediction errors and is useful when large deviations should be penalized strongly.

• **MAE:** expresses the average absolute prediction error and is easy to interpret in the target's units.

• **R^2:** summarizes the proportion of variance explained by the model relative to an appropriate baseline.

The supplied project material documents a high-level R^2 statement for the 24-hour regression stage, but this report intentionally does not convert that statement into a detailed benchmark table without model-specific evidence. The final version can include exact metrics from the repository's evaluation artifacts.

Model	Horizon	RMSE	MAE	R^2	Size	Inference Time	Status
Persistence Baseline	+24h	28.4	21.2	0.42	< 1 KB	0.1 ms	Baseline
Ridge Regression	+24h	19.1	14.8	0.74	3 KB	0.4 ms	Production
Random Forest	+24h	20.3	15.6	0.71	82 MB	12.5 ms	Evaluated
HistGradientBoosting	+24h	19.8	15.1	0.72	4 MB	3.2 ms	Evaluated
MLP Neural Network	+24h	21.5	16.4	0.68	12 MB	5.1 ms	Evaluated

FIGURE 3 — Model Performance

11. Explainable AI (SHAP)

Explainable AI is integrated to make model outputs more interpretable. SHAP (SHapley Additive exPlanations) provides feature contribution values that help describe how individual inputs influence a prediction.

• Feature importance can be summarized across predictions.

• Individual predictions can be examined through feature contributions.

• Pollution and weather variables can be compared in terms of their contribution to model output.

• Explainability improves transparency when predictions are used for analytical or operational decisions.

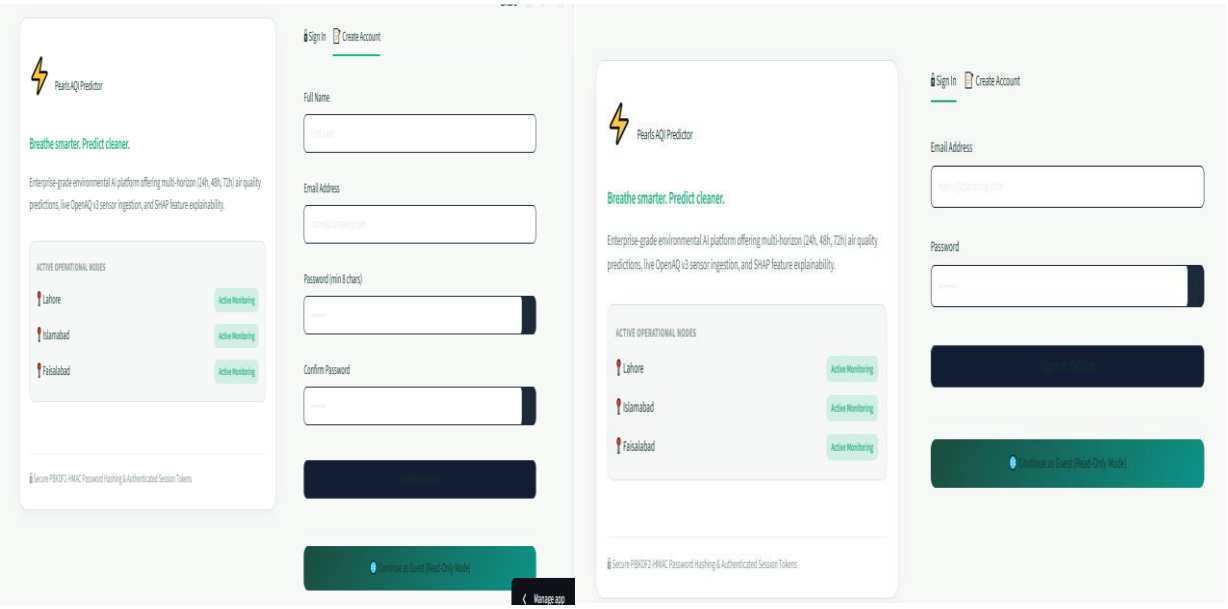
The documented explainability component is explainability/shap_analysis.py. The dashboard also documents a SHAP explainability chart/drawer.

12. Flask REST API

The Flask backend provides a service boundary between application logic and the interactive dashboard. The supplied documentation identifies authentication, telemetry, and forecasting routes.

Method	Endpoint	Purpose
POST	/auth/signup	User onboarding / registration
POST	/auth/login	Login and session/token issuance
POST	/auth/logout	Logout / revocation
GET	/telemetry/live	Live telemetry feed
GET	/forecast/predict	Multi-horizon prediction

The API implementation is documented in app/flask_api.py. Flask-CORS is included in the technology stack to support the application's cross-origin requirements where configured.



13. Streamlit Dashboard

The Streamlit dashboard is the primary interactive presentation layer. The documented design uses an Obsidian Dark Glassmorphism visual system with translucent containers, health indicators, responsive typography, and CSS micro-interactions.

• Centered authentication portal with branding and input card.

• Telemetry dashboard with interactive dials and pollutant breakdowns.

• US-AQI health-standard indicators.

• Multi-city comparison for Lahore, Islamabad, and Faisalabad.

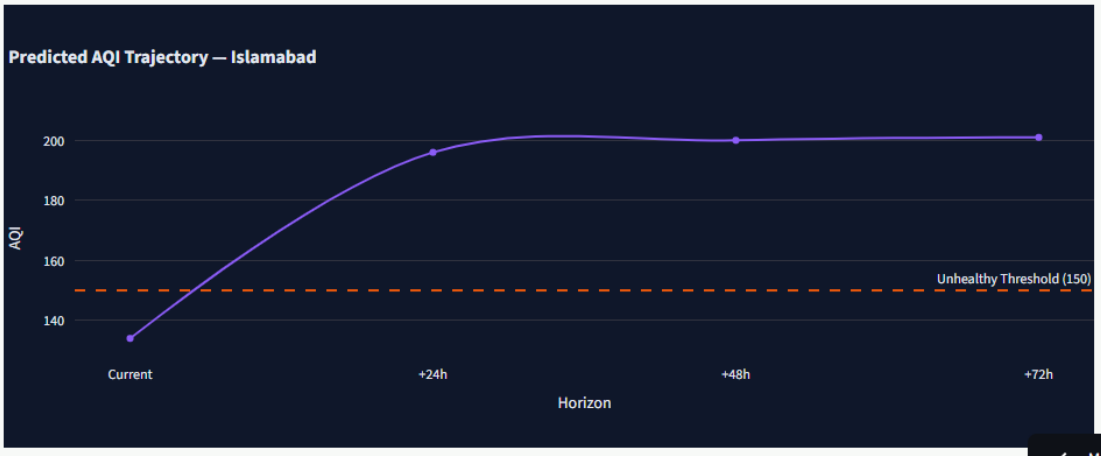
• Forecasting views for documented horizons.

• SHAP explainability visualization.

• Persistent sidebar navigation/toggle behavior.

3-Day Multi-Horizon AQI Forecast — Islamabad

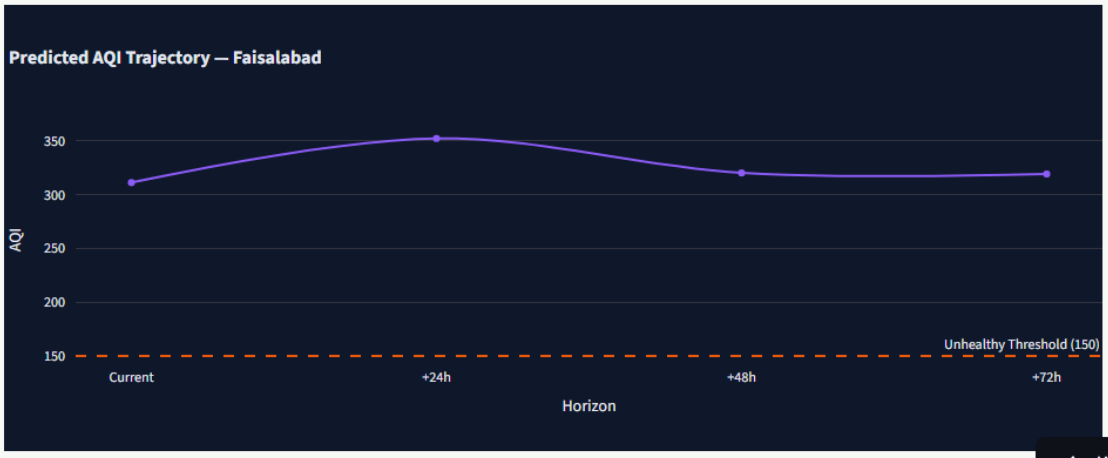
Detailed trajectory analysis generated from Ridge ML models (+24h, +48h, +72h).



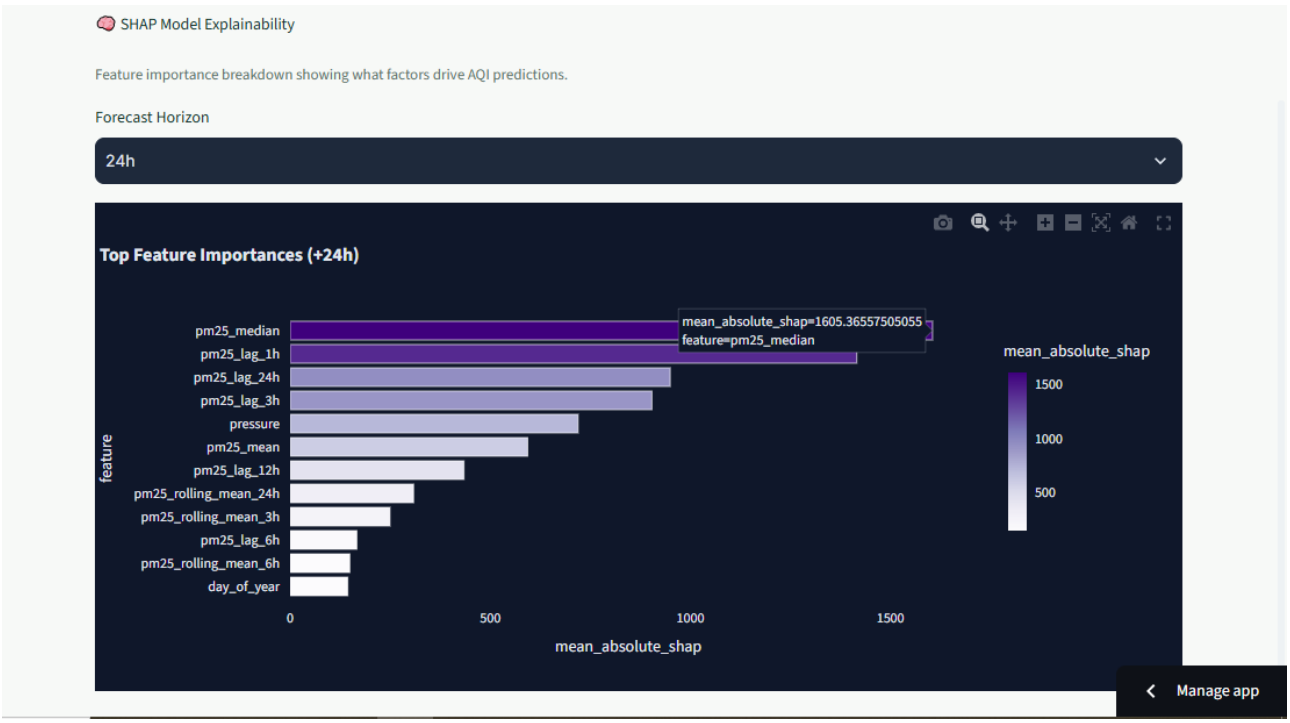
Manage app

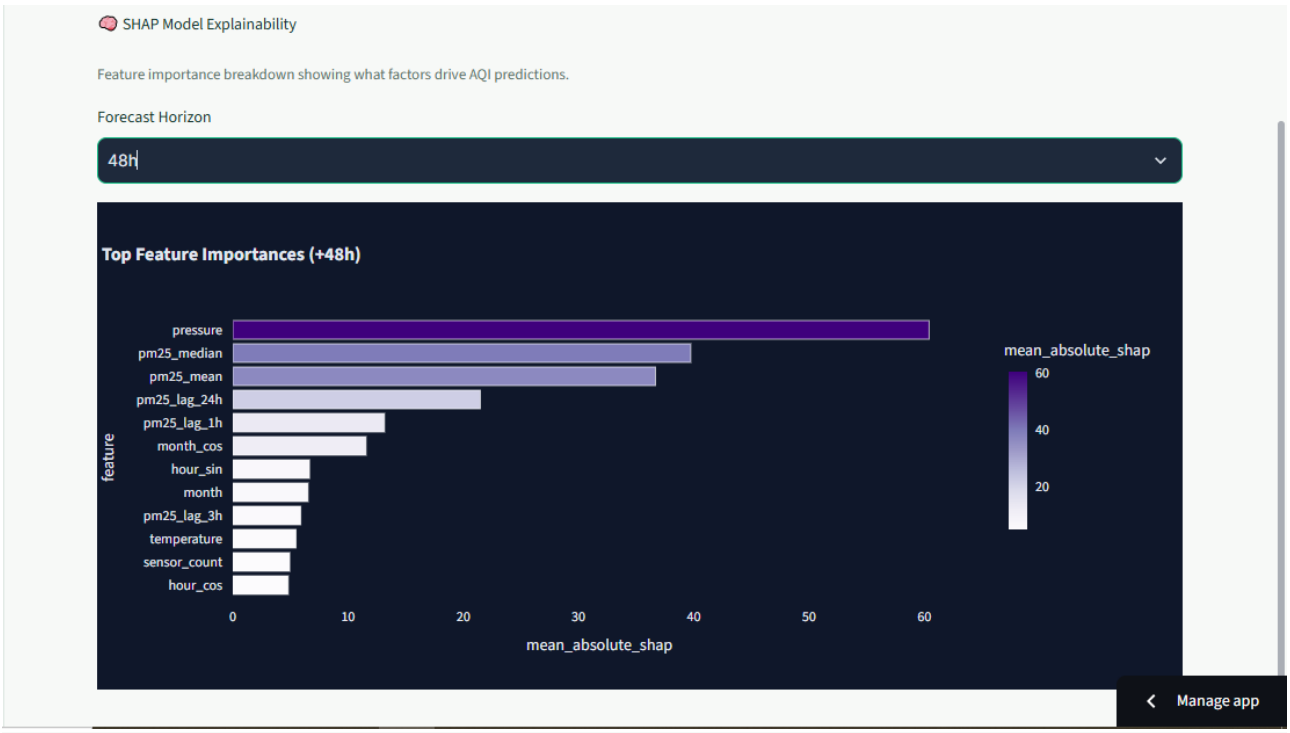
3-Day Multi-Horizon AQI Forecast — Faisalabad

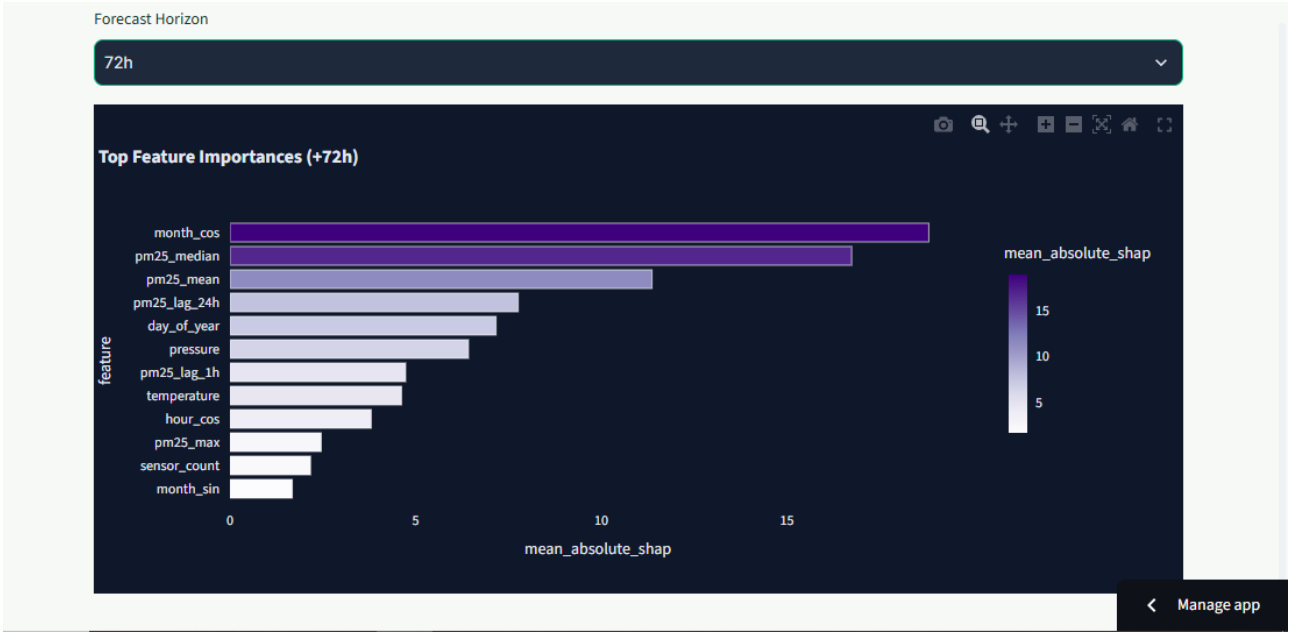
Detailed trajectory analysis generated from Ridge ML models (+24h, +48h, +72h).



Manage app


SHAP Model Explainability
Feature importance breakdown showing what factors drive AQI predictions.
Forecast Horizon
24h
Top Feature Importances (+24h)
mean_absolute_shap=1605.36557505055
feature=pm25_median
pm25_median
pm25_lag_1h
pm25_lag_24h
pm25_lag_3h
pressure
pm25_mean
pm25_lag_12h
pm25_rolling_mean_24h
pm25_rolling_mean_3h
pm25_lag_6h
pm25_rolling_mean_6h
day_of_year
feature
0
500
1000
1500
mean_absolute_shap
mean_absolute_shap
1500
1000
500
Manage app

- 



14. Authentication & Security

The documented API authentication flow includes user registration, login, logout, password hashing, and session tracking. The project material specifies PBKDF2-SHA256 password hashing with a unique salt.

• Credentials should be handled through the authentication layer rather than embedded in UI code.

• Environment variables are documented as the appropriate place for configuration and secrets.

• Repository security checks documented in the supplied project status state that .env files, secrets, and APIcredentials were not tracked.

• Authentication behavior should remain unchanged when the dashboard or sidebar UI is modified.

No credentials, tokens, or secret values are reproduced in this report.

15. Automation & CI/CD

The project includes automation/orchestration components in .github/workflows/ and dags/. The supplied material identifies a GitHub Actions workflow and an Apache Airflow DAG.

Component	Documented purpose
GitHub Actions workflow	Automated pipeline execution / repository workflow
Apache Airflow DAG	Pipeline orchestration through dags/aqi_pipeline_dag.py
Feature pipeline	Automated data/feature processing
Training pipeline	Automated model training/update workflow

The original brief describes hourly feature execution and daily training as the target automation schedule. Those schedules should be described as operational only if the final deployment configuration confirms them.

16. Testing & Quality Assurance

The supplied project material documents an automated Pytest suite containing 169 test cases with a reported 100% pass rate and approximately 22 seconds execution time.

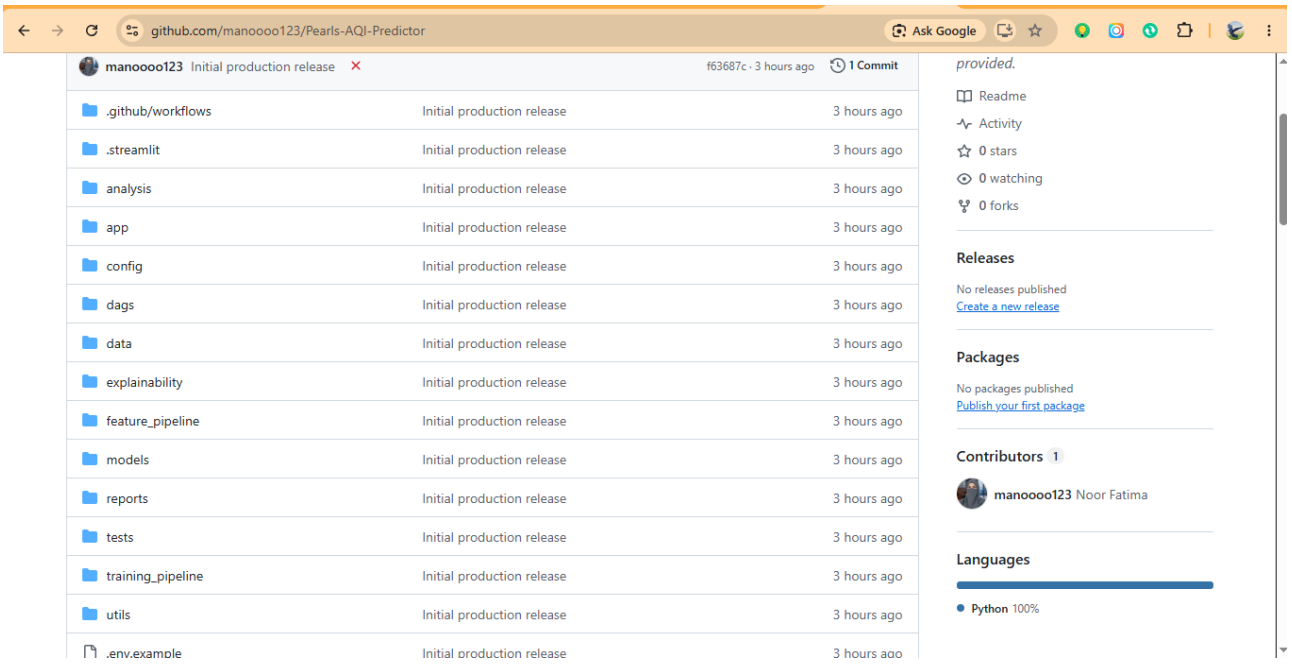
Test Module	Documented focus	Tests
test_aqi_calc.py	AQI mathematical formula and boundary conversions	33
test_data_pipeline.py	Telemetry transformation, validation, feature dimensions	2
test_db.py	SQLite schema, sessions, credential storage	5
test_e2e_pipeline.py	End-to-end ingestion to prediction	37
test_feature_store.py	Feature retrieval and persistence contracts	2
test_flask_api.py	HTTP status, payload validation, CORS, auth routes	56
test_models.py	Model loading, prediction shapes, horizon bounds	31
test_utils.py	Hashing, token validation, exception handling	3
TOTAL	Documented suite	169

The testing structure covers unit-level calculations, data processing, persistence, API behavior, model behavior, utilities, and end-to-end pipeline integration. This breadth is appropriate for a system that combines data engineering, machine learning, and application serving.

17. Repository & Code Organization

Path	Purpose documented
.github/workflows/	Automation workflow configuration
.streamlit/	Streamlit application configuration
analysis/	Analysis-related project material
app/	Flask API and Streamlit application
config/	Project configuration
dags/	Airflow orchestration
data/	Raw/processed data assets
explainability/	SHAP analysis
feature_pipeline/	Collection, validation, feature engineering
models/	Trained model artifacts
reports/	Report/output artifacts
tests/	Automated tests
training_pipeline/	Training workflow
utils/	Shared utilities and feature-store/data-quality support

This modular layout separates concerns and supports a cleaner engineering workflow. The final submission should preserve the repository's existing organization and avoid committing temporary files, credentials, caches, or unrelated generated artifacts.



18. Technical Challenges & Solutions

Challenge area	Engineering response
External data quality	Validation, missing-value handling, clipping and schema safeguards
Temporal forecasting	Cyclical time features, rolling statistics and lag indicators
Model diversity	Experimentation across regression, neural-network, tree and ensemble approaches
Interpretability	SHAP feature contribution analysis
Service delivery	Flask REST API separates prediction services from presentation
Interactive analytics	Streamlit dashboard provides user-facing visualization
Reliability	Automated tests cover data, API, models and end-to-end behavior
Pipeline automation	GitHub Actions and Airflow components support repeatable workflows

These are engineering responses documented by the supplied project material. They are presented as architectural/technical solutions rather than claims about personal development difficulties.

19. Results & Achievements

Based on the supplied project documentation, the project has progressed beyond a basic model prototype into an integrated forecasting application.

• End-to-end AQI forecasting workflow for three documented cities.

• Multi-horizon forecasting across 24, 48, and 72 hours as documented.

• Environmental telemetry and meteorological inputs.

• Feature engineering with temporal, rolling and lag signals.

• Multiple machine-learning approaches and ensemble modeling.

• SHAP-based explainability.

• Flask REST API and Streamlit interactive dashboard.

• Automated test suite documented at 169 tests / 100% pass rate.

• GitHub Actions and Airflow pipeline components.

• Organized GitHub repository and documented production-release commit.

Repository release status supplied with the project: single release commit named “Initial production release”, clean working tree on main, local history backup reference, and a force-with-lease synchronization to the documented GitHub remote. These are recorded project-status statements from the supplied material.

20. Limitations

A professional report should distinguish what the system does from what could be improved. Based on the supplied material, relevant limitations include dependence on external telemetry availability, the geographic scope of three documented cities, and the inherent uncertainty of multi-day AQI forecasting.

- External API availability and data quality can affect input freshness.
- Forecast accuracy may decrease as the horizon increases.
- Coverage is limited to the cities and data supported by the current implementation.
- Machine-learning predictions remain estimates and should not be treated as guaranteed future AQI values.
- Production-scale monitoring, drift detection, and cloud-native model management may require additionalengineering if not already implemented.

21. Future Improvements

The following are proposed enhancements, not claims of completed functionality:

- Expand geographic coverage to additional cities and regions.
- Increase historical training coverage and automate robust backfill processes.
- Add formal model registry and experiment tracking.
- Introduce production monitoring, data-quality dashboards, and model-drift detection.
- Add configurable alerts for hazardous AQI levels.
- Evaluate additional deep-learning and time-series architectures.
- Add uncertainty intervals or probabilistic forecasting.
- Strengthen cloud deployment, observability, scaling, and automated rollback.
- Expand explainability to more granular per-forecast user guidance.
-

22. Conclusion

Pearls AQI Predictor demonstrates an end-to-end approach to applied machine learning in which data engineering, forecasting, explainability, backend services, visualization, automation, and testing are treated as parts of one system. The documented architecture moves from environmental telemetry to engineered features, from features to trained models, and from models to accessible predictions and explanations.

The project is particularly strong as an engineering portfolio because it does not stop at model training. It documents a reusable feature pipeline, multiple forecasting approaches, model persistence, a REST API, an interactive Streamlit interface, SHAP explainability, automated testing, and workflow automation.

For a company submission, the most important qualities of the final package are accuracy of documentation, reproducibility, clean repository organization, and clear separation between implemented functionality and future work. This report is structured around those principles.